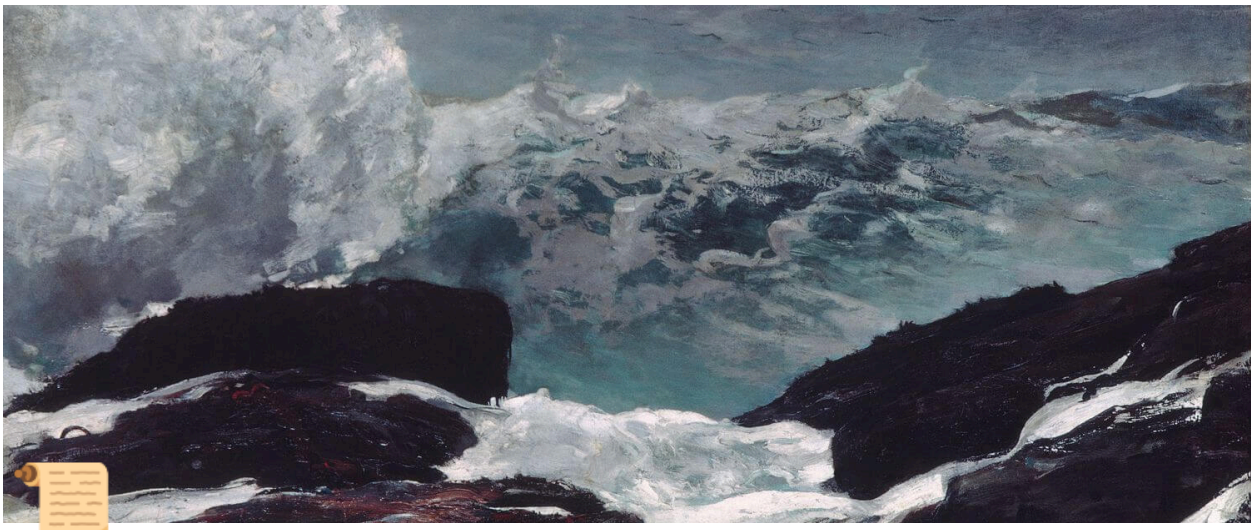

Tavi Hackathon (Founding Technical / Product)

📬 Our Mission

Tavi is the AI-native managed marketplace for ordering, managing, and paying for any trade. There are two sides to our marketplace:

- **Facility managers** at multi-site commercial locations who seek to place service orders (e.g., fixing a leaking pipe at a Walmart)
- **Service Vendors** who perform the work, and are rated by a combination of their (i) price (ii) service quality and (iii) availability to perform the work

---

💻 Product Flow & the Task

The hackathon task: build your own version of Tavi and show us how it completes a work order from end-to-end in a real-world setting. Here is the basic flow of our product for reference:

- Work order intake: Facility managers must be able to submit a work order (e.g., I need this plumbing job done for my Dallas location on 123 main street

for $250 by a reputable vendor on Tuesday at 5pm) through a multi-modal natural language interface (e.g., ideally voice conversation with an agent, or fallback chat interface which then populates a form):

  - Example of a legacy provider providing work order intake through a non-natural language interface: https://academy.getmaintainx.com/pages/training-hub.

- Vendor discovery: After the work order is specified for a relevant trade (e.g., lawncare, electrical, plumbing), Tavi programmatically seeks out all service providers within 20 miles of the service location and assigns them a quality score based on aggregating public sources (e.g., Google Reviews, Better Business Bureau, License information [in practice we aggregate across 21+ sources])

- Vendor contact / auctioning: After identifying relevant vendors and their contact information, we employ a multi-modal approach to agentically contact service vendors over phone, email, and text in order to present, auction the job, and dispatch quotes.
  - Our system is set-up as a human-in-the-loop command center, where agents complete leg work and humans intervene where they notice snags. All modalities of contact (phone, email, text) are in a single stream of context to inform future responses.
  - Think of an agent living at the intersection of every job x every prospective vendor for the job which dynamically stores a state machine of where the vendor is in the process (e.g., prospecting, negotiating, dispatching, etc) & is able to take discrete actions (e.g., schedule a site visit, request and verify insurance)
  - **This is the bulk of our engineering work, hence you should spend most time here**

We don't expect that all pieces can be completed within the deadline. Focus on elements that show off your technical aptitude, hustle, and taste, while presenting

a coherent full-stack app that can run the process end-to-end.

Important: We are not seeking free labor - your code/IP will be owned by you and will not be integrated into our production codebase

---

🥇 Evaluation:

#1 Technical aptitude - your relevant full-stack / data architecture skills & working with agents

#2 Hustle - your ability to power through significant feature rollouts on tight timelines

#3 Taste - your conception of intuitive workflows that will survive real-world settings

🔖 Guidelines:

- Bring your own API keys & infra + work out of your own repo. We will reimburse up to $100 in direct expenses (send time-stamped billing screenshots).
- Please give access to the repo (on Github or the like) & also setup/ run instructions after submission, rather than just a deployed prod link
- Leveraging AI tools for development highly encouraged. Please use source-code first tools like Claude Code / Cursor rather than end-to-end tools like Replit or Lovable.
    - If using Claude Code/ Codex, please plan to export your prompt history - this helps us understand your problem solving approach
- Our internal stack: postgres / fastapi / react / nextjs (don't have to conform to this, but preferred that you can work in this environment)
- Call or text RK/DG at any time to discuss technical blockers or questions. Plan on:
    - Finding 60 minutes to co-work & screenshare at some point to walk though a feature you are building live

- 60 min demo / technical rundown after you’re done in ~48 hours